

Practical extraction and reporting language (Perl)

Nathan Warner



Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

1	Perl	2
2	Comments	4
3	Command line options	5
4	Variables	6
5	Arrays	8
6	Hashes	10
7	Special variables	12
8	Operators and functions	13

Perl

- **Practical extraction and reporting language (perl):** Perl is a high-level, general-purpose, interpreted, dynamic programming language. Though Perl is not officially an acronym, **Practical Extraction and Reporting Language** is commonly used as a **backronym**.
- **use version pragma:** The use VERSION pragma in Perl enforces a minimum version required to run a script and automatically enables modern language features

You should place this declaration at the very top of your file. Modern Perl code uses the vX.Y v-string format rather than old-style decimals:

```
0 use v5.36
```

Versions v5.10 and higher automatically activate the feature pragma for that release. For example, use v5.36; is equivalent to loading use feature ':5.36';. This gives you immediate access to built-in keywords like say, state, and updated signatures without typing extra code

Declaring use v5.11 or higher automatically turns on the strict pragma (use strict;) lexically across your file.

Declaring use v5.35 or higher automatically turns on the warnings pragma (use warnings;

- **use feature:** In Perl, the use feature pragma allows you to enable newer syntactic features and capabilities without breaking backward compatibility for older programs. Because it is a lexical pragma, any feature you enable is only active from the line it is declared until the end of its enclosing code block.
 - **Individual Features:** Pass a list of feature names to the pragma.

```
0 use feature 'say';
1 say "Hello, world!"; # Appends a newline
```

- **Feature bundles:** Load all features made available in a specific Perl release by prefixing the version with a colon

```
0 use feature ':5.36'
```

- **Implicit loading:** Requiring a minimum Perl version automatically enables the feature bundle for that version
- **Command Line Execution:** Use the -E flag instead of -e to quickly test code snippets with all features enabled for your current Perl version.

```
0 perl -E 'say "Fast one-liner testing!'"
```

- **Using the diagnostics pragma:** Another pragma that you may find useful is the diagnostics pragma. This translates the normally terse diagnostics emitted from the perl compiler and the perl interpreter into much more useful ones

```
0  use diagnostics;
```

The diagnostics pragma makes your warnings much more verbose, and it slows the start-up time of your script considerably. You should remove it before putting your code into production.

Comments

- **Standard comments:** Uses a hashtag

```
0  # Comment
```

- **Block comments:** We can also use Plain Old Documentation (POD)

```
0  =begin comment
1  ...
2  ...
3  =end comment
4
5  =cut
```

=cut resumes Perl code execution. "Stop ignoring this text; switch back to executable Perl code."

=end comment closes a =begin comment region. "This specific comment block is done, but continue treating subsequent text as documentation."

- **__END__**: You can signal to Perl the end of your program by using the special **__END__** tag. Anything below **__END__** will be ignored by Perl.

```
0  #!/usr/bin/perl
1
2  print "Hello world";
3
4  __END__
```

Command line options

- **Warnings:** Turn on warnings with -w

```
o  #!/usr/bin/perl -w
```

Variables

- **Pragmatic information (pragma):** A code pragma (short for "pragmatic information") is a special instruction or command embedded in source code. It tells a compiler, interpreter, or assembler how to process the program, but it does not change the actual logic or output of the software itself.
- **Note about variables:** In Perl, it is not necessary to declare your variables before you begin. You can summon a variable into existence simply by using it, and it will be globally available to any routine in your program. If you're used to programming in C or any of a number of other languages, this may seem odd and even dangerous to you. This is indeed the case. That's why you want to use the strict pragma.
- **Variable sigils (prefixes):** In Perl, variable prefixes are called sigils, and they immediately identify the data type of the variable.
 - \$ used for scalars
 - @ used for arrays
 - % used for hashes. A hash is an unordered collection of key-value pairs (associative arrays).
- **my, our, and state:** In Perl, my declares a truly private lexical variable, state creates a persistent static variable that initializes only once, and our creates a temporary shortcut to a global package variable

A lexical variable is a variable that is available only within a specific, textually defined part of a program, such as a function, a block, or a file.

- **The strict pragma:** Once we use the strict pragma, we have to explicitly declare new variables with scope keywords my, our, or state.

```
0  #!/usr/bin/perl -w
1  use strict;
2
3  my $x = 50; # Okay
4  $y = 100; # Bad
```

- **Variables with name a or b:** \$a and \$b are special variables in Perl. they are implicitly used by Perl's built-in sort function. Inside the sort block, Perl automatically places the two values being compared into \$a and \$b. Consequently, strict 'vars' specifically permits \$a and \$b without requiring my
- **Scalars:** A scalar is a single item of data

```
0  my $name = "Arthur";
1  my $whoami = 'Just Another Perl Hacker';
2  my $meaning_of_life = 42;
3  my $number_less_than_1 = 0.000001;
4  my $very_large_number = 3.27e17; # 3.27 by 10 to the power of
   ↪ 17
```

A scalar can be text of any length, and numbers of any precision (machine dependent, of course). Perl doesn't need us to tell it what type of data we're going to put into the scalar. In fact, Perl doesn't care if the type of data in the scalar changes throughout your program. Perl magically converts between them when it needs to

- **Single vs double quotes:** In Perl, quotes are required to distinguish strings from the language's reserved words or other expressions. Either type of quote can be used, but there is one important difference: double quotes can include other variable names inside them, and those variables will then be interpolated

```
0 my $x = 50;  
1 print "$x";
```

- **Special characters:** Special characters, such as the `\n` newline character, are only available within double quotes. Single quotes will fail to expand these special characters just as they fail to expand variable names
- **String concatenation:** Uses the dot notation

```
0 print "Hello, " . "World!";
```

- **Bash style interpolation:**

```
0 my $x = "Hello";  
1 print "${x}, World";
```


Arrays

- **Creating arrays:** Arrays are initialised by creating a comma separated list of values:

```
0 my @arr = (1,2,3);
```

Perl's arrays, like those in many other languages, are indexed from zero

- **Accessing:** As you would expect

```
0 print $fruits[0];
```

We use the dollar sign here because the indexing produces a scalar.

- **Slicing:** If we wanted to only deal with a portion of the array, we use what we call an array slice. These are written as follows:

```
0 @arr # entire array
1 @arr[1,3]
2 @arr[0..2]
```

- **Negative indexing:**

```
0 print $arr[-1] # 3
```

- **Array size:**

```
0 print $#arr + 1;
```

Note that `$#arr` gets the index of the last element. We can also use

```
0 my $size = @arr;
1 print my $size = @arr;
```

- **qw// (quote words):** It takes whitespace separated words and turns them into a list

```
0 my @arr = qw/hello , world/
```

Your list can stretch over multiple lines, and your delimiter doesn't need to be a slash. Whatever punctuation character that you place after the `qw` becomes the delimiter. So if you prefer parentheses over slashes, that's no problem at all

- **Printing out arrays**

```

0  my @arr = (1,2,3);
1  print @arr;
2  print join(", ", @arr);
3  print "@arr"

```

The first method takes advantage of the fact that print takes a list of arguments to print and prints them out sequentially. The second uses join() which joins an array or list together by separating each element with the first argument. The third option uses double quote interpolation and a little bit of Perl magic to pick which character(s) to separate the words with.

- **Context:** There's a term you've heard used just recently but which hasn't been explained: context. Context refers to how an expression or variable is evaluated in Perl. The two main contexts are:
 - Scalar context
 - List context

Scalar variables are always evaluated in scalar context, however arrays and hashes can be evaluated in both scalar contexts (when we treat them as scalars) and list contexts (when we treat them as arrays and hashes). Scalar variables are always evaluated in scalar context, however arrays and hashes can be evaluated in both scalar contexts (when we treat them as scalars) and list contexts (when we treat them as arrays and hashes).

Many things in Perl are very specific about which context they require and will force lists into scalar context when required. For example the + (plus or addition) operator expects that its two arguments will be scalars. Hence:

```

0  my @a = (1,2,3);
1  my @b = (4,5,6,7);
2  print @a + @b;

```

will print 7. There's also a third type of context, the null context, where the result of an operation is just thrown away. This usually isn't discussed, because by its very definition we don't care about what result is returned

- **Lists:** A list is an unnamed array.

```

0  print ("Hello", ",", " ", "World", "\n");

```

If you come across something that wants a LIST, you can either give it the elements of list as in the first example above, or you can pass it an array by name. If you come across something that wants an ARRAY, you have to actually give it the name of an array. Examples of functions which insist on wanting an ARRAY are push() and pop(), which can be used for adding and removing elements from the end of an array

Hashes

- **Hashes:** A hash is a two-dimensional array which contains keys and values, they're sometimes called "associative arrays", or "lookup tables". Instead of looking up items in a hash by an array index, you can look up values by their keys.

Hashes are initialised in exactly the same way as arrays, with a comma separated list of values:

```
0 my %monthdays = ("January", 31, "February", 28, "March", 31,  
  ↪ ...);
```

Or,

```
0 my %monthdays = (  
1   "January" => 31,  
2   "February" => 28,  
3   "March" => 31,  
4   ...  
5 );
```

The => operator is syntactically the same as the comma, but is used to distinguish hashes more easily from normal arrays. It's pronounced "fat comma", or less often "fat arrow". It does have one difference, you don't need to put quotes around a bare word immediately before the => operator as these are always treated as strings:

- **Reading hash values:**

```
0 print $monthdays{"January"}; # prints 31
```

Bare words inside the braces of the hash-lookup will be interpreted in Perl as strings, so usually you can drop the quotes:

- **Creating hash elements:** You can also create elements in a hash on the fly:

```
0 my %monthdays = ();  
1 $monthdays{"January"} = 31;  
2 $monthdays{February} = 28;  
3 ...
```

- **Changing hash values:** To change a value in a hash, just assign the new value to your key:

```
0 $pizza_prices{small} = '$6.00'; # Small pizza prices have  
  ↪ gone up
```

- **Deleting hash values:** Use the delete function

```
0 delete($pizza_prices{medium}); # No medium pizzas anymore.
```

- **Finding out the size of a hash:** Strictly speaking there is no equivalent to using an array in a scalar context to get the size of a hash. If you take a hash in a scalar context you get back the number of buckets used in the hash, or zero if it is empty. This is only really useful to determine whether or not there are any items in the hash, not how many.

If you want to know the size of a hash, the number of key-value pairs, you can use the `keys` function in a scalar context. The `keys` function returns a list of all the keys in the hash.

```
0 my $size = keys %monthdays;  
1 print $size;
```

Special variables

- **Intro:** Perl has many special variables. These are used to set or retrieve certain values which affect the way your program runs. For instance, you can set a special variable to turn interpreter warnings on and off (`$^W`), or read a special variable to find out the command line arguments passed to your script (`@ARGV`).
- `$_`: Represents the current thing that your Perl script's working with — often a line of text or an element of a list or hash. It can be set explicitly, or it can be set implicitly by certain looping constructs (which we'll look at later).

The special variable `$_` is often the default argument for functions in Perl. For instance, the `print()` function defaults to printing `$_`.

```
0  $_ = "Hello, world!\n";  
1  print;
```

- **@ARGV:** Command line args end up in this special array.

```
0  # ./main.pl arg1 arg2  
1  print "{ARGV}" arg1 arg2  
2  print my $argc = @ARGV; 2
```

- **%ENV:** This hash contains the names and values of environment variables.

Operators and functions

- Operators

- **Arithmetic:** +, -, *, /, %, ** (exp), +=, -=, *=, /=, %, **=

Note: In fact, the shortcut operators += and its siblings can be used for just about any operator.

We also have post/pre increment/decrement.

- **String:** . (concat), x (repeat)

```
0  "hello" . "world";  
1  "hello" x 3;
```